Q1) Answer the following questions. [DRL Exam Question 1 ; 22-July-2023]

(a) Consider the problem of learning optimal values that control traffic signals at a zebra crossing. The amount of time pedestrians wait for the signal (red), and the amount of time they get while crossing the road (green) are adequate to control the traffic. Provide a MDP formulation to this scenario with all the necessary details.Explain all the design choices. [3.5 M]

(b) Write the bellman state-value equation for your MDP in (a). Show the corresponding back-up diagram. [2.0 M]

(c) For the given formulation, which of the approaches to solving MDP is appropriate. Explain. [1.0 M]

(d) Explain what happens if all the rewards in (a) are multiplied by 10? [1.0 M]

Answer:

(a)

To formulate the problem of controlling traffic signals at a zebra crossing as a Markov Decision Process (MDP), we need to define the following components: states, actions, transition probabilities, rewards, and the discount factor. Let's go through each of these components:

States (S)
The state should represent the current status of the traffic signal and possibly other relevant information such as the number of waiting pedestrians and vehicles.

1. `S = (T, P, V)`: where
   - $T \in \{ \text{Red, Green} \}$ represents the current state of the traffic signal.
   - P is the number of pedestrians waiting to cross.
   - V is the number of vehicles waiting to pass.

Actions (A)
The actions should represent the possible changes to the traffic signal. For simplicity, let's assume the actions are:

1. `A = {Switch to Red, Switch to Green, Maintain Current State}`

Transition Probabilities (P)
The transition probabilities represent the likelihood of moving from one state to another given a particular action. This involves considering how pedestrian and vehicle arrivals change over time and how the traffic signal changes.

1. If the action is `Switch to Green`:
   - The signal state changes to Green.
   - Pedestrians start crossing, reducing P based on the crossing rate.
   - Vehicles accumulate as they wait, increasing V based on the arrival rate.

2. If the action is `Switch to Red`:
   - The signal state changes to Red.
   - Vehicles start passing, reducing V based on the passing rate.
   - Pedestrians accumulate as they wait, increasing P based on the arrival rate.

3. If the action is `Maintain Current State`:
   - If Green, pedestrians continue crossing and vehicles accumulate.
   - If Red, vehicles continue passing and pedestrians accumulate.

The exact transition probabilities would be determined by traffic patterns, pedestrian crossing speeds, and vehicle arrival rates, which can be modeled or estimated based on real-world data.

Rewards (R)
The rewards should reflect the goal of minimizing pedestrian waiting time and balancing the waiting time for vehicles. We can assign a negative reward (or cost) for each time step that pedestrians or vehicles wait.

1. `R(S, A)`:
   - A negative reward proportional to the number of waiting pedestrians P and vehicles V:
     $$
     R(S, A) = - (\alpha \cdot P + \beta \cdot V)
     $$
     where α and β are weights reflecting the relative importance of pedestrian and vehicle waiting times.

Discount Factor (γ)
The discount factor $\gamma \in [0, 1]$ determines the importance of future rewards. A typical choice for traffic signal control could be around 0.9, balancing immediate rewards with future considerations.

MDP Formulation Summary
- **States (S)**: $S = (T, P, V)$
- **Actions (A)**: $A = \{\text{Switch to Red, Switch to Green, Maintain Current State}\}$
- **Transition Probabilities (P)**: Defined based on pedestrian and vehicle dynamics
- **Rewards (R)**: $R(S, A) = - (\alpha \cdot P + \beta \cdot V)$
- **Discount Factor (γ)**: Typically around 0.9

Design Choices Explanation
1. **State Representation**: Including the signal state, pedestrian count, and vehicle count captures the essential dynamics affecting the system's performance.
2. **Action Set**: Simple actions of switching or maintaining the signal state cover the basic control options available in a real-world traffic signal system.
3. **Transition Probabilities**: Reflect the natural progression of pedestrian and vehicle dynamics, providing a realistic model of the environment.
4. **Reward Function**: Directly targets the objective of minimizing waiting times, making it suitable for optimizing overall traffic flow efficiency.
5. **Discount Factor**: Balances short-term and long-term impacts, which is crucial in traffic management to avoid short-sighted decisions.

By solving this MDP using standard techniques like value iteration or policy iteration, we can derive an optimal policy for controlling the traffic signals to minimize waiting times for both pedestrians and vehicles.

(b)

The Bellman state-value equation for the MDP, which describes the value of a state S under an optimal policy, is given by:

$$V(S) = \max_{A} \left[R(S, A) + \gamma \sum_{S'} P(S'|S, A) V(S') \right]$$

where:
- $V(S)$ is the value of state S.
- A is the set of possible actions.
- $R(S, A)$ is the reward for taking action A in state S.
- γ is the discount factor.
- $P(S'|S, A)$ is the transition probability of reaching state S' from state S given action A.

For our specific MDP, the Bellman equation can be written more explicitly as:

$$V(T, P, V) = \max_{A \in \{\text{Switch to Red, Switch to Green, Maintain Current State}\}} \left[R((T, P, V), A) + \gamma \sum_{(T', P', V')} P((T', P', V') | (T, P, V), A) V(T', P', V') \right]$$

Let's break this down:

1. **State (T, P, V)**:
   - T is the traffic signal state (Red or Green).
   - P is the number of waiting pedestrians.
   - V is the number of waiting vehicles.

2. **Action A**:
   - Possible actions are Switch to Red, Switch to Green, Maintain Current State.

3. **Reward $R((T, P, V), A)$**:
   - The reward is calculated as:
     $$R((T, P, V), A) = - (\alpha \cdot P + \beta \cdot V)$$

4. **Transition Probabilities $P((T', P', V') | (T, P, V), A)$**:
   - Transition probabilities depend on the action taken and the current state.
   - These probabilities are influenced by factors such as pedestrian crossing rates, vehicle arrival rates, and the current signal state.

The Bellman state-value equation takes into account the immediate reward of taking action A in state (T, P, V) and the expected value of the next state (T', P', V'), discounted by γ.

Example Calculation
For a specific state (T, P, V) and action Switch to Green:

$$V(\text{Red}, P, V) = \max_{A} \left[R((\text{Red}, P, V), A) + \gamma \sum_{(\text{Green}, P', V')} P((\text{Green}, P', V') | (\text{Red}, P, V), \text{Switch to Green}) V(\text{Green}, P', V') \right]$$

Here, we consider the reward for switching the signal to green, the probability of transitioning to different states \((\text{Green}, P', V') \) given this action, and the value of those states. This process is repeated for each action to find the one that maximizes the expected value, thus determining the optimal policy.

(c)

Given the formulation of the MDP for controlling traffic signals at a zebra crossing, several approaches could be considered for solving the MDP. The choice of the appropriate approach depends on various factors, including the size of the state and action spaces, the availability of a model of the environment, and computational resources. Here are some common approaches and an explanation of their suitability for this problem:

1. **Dynamic Programming (DP) Approaches**
Dynamic Programming approaches require a complete model of the environment (i.e., transition probabilities and rewards) and are suitable for problems with a manageable state and action space. The two main DP methods are:

Value Iteration
- **Value Iteration** is an iterative algorithm that updates the value of each state based on the Bellman equation until convergence.
- **Suitability**: If the state and action spaces are relatively small and the transition probabilities and rewards can be accurately modeled, value iteration is a good choice. It provides a straightforward way to compute the optimal policy.

Policy Iteration
- **Policy Iteration** involves iterating between policy evaluation (computing the value of a policy) and policy improvement (updating the policy based on the value function).
- **Suitability**: Like value iteration, policy iteration is suitable if the state and action spaces are manageable and the model of the environment is available. It can converge faster than value iteration in some cases.

2. **Reinforcement Learning (RL) Approaches**
Reinforcement Learning approaches are suitable when the model of the environment is not fully known and the agent needs to learn the optimal policy through interaction with the environment. Common RL methods include:

Q-Learning
- **Q-Learning** is a model-free RL algorithm that learns the value of state-action pairs (Q-values) through exploration and exploitation.
- **Suitability**: If the state and action spaces are large, and the transition probabilities and rewards are not fully known or are difficult to model accurately, Q-Learning is a good choice. It can handle large state spaces by using function approximation methods such as neural networks (Deep Q-Learning).

SARSA (State-Action-Reward-State-Action)
- **SARSA** is another model-free RL algorithm similar to Q-Learning but updates Q-values based on the action actually taken by the policy.
- **Suitability**: Like Q-Learning, SARSA is suitable for large state spaces and unknown environments. It can also be combined with function approximation techniques.

3. **Approximate Dynamic Programming (ADP) and Deep Reinforcement Learning (DRL)**
For very large state and action spaces, traditional DP and RL methods may not be feasible due to the "curse of dimensionality". In such cases, approximate methods using function approximation (e.g., neural networks) are appropriate.

Deep Q-Networks (DQN)
- **DQN** uses a neural network to approximate the Q-value function, allowing it to handle large state spaces.
- **Suitability**: If the state space is very large (e.g., high-dimensional representations of the environment), DQN is suitable. It leverages deep learning to approximate the Q-value function efficiently.

Recommended Approach for Traffic Signal Control
Considering the characteristics of the traffic signal control problem:
- **State Space**: The state space (T, P, V) can become large if the number of waiting pedestrians and vehicles is represented with high granularity.
- **Action Space**: The action space is relatively small.
- **Model Availability**: Transition probabilities can be estimated, but exact models may not be easily obtainable.
- **Learning from Interaction**: The system can learn from interaction with the environment (e.g., through simulation or real-world data).

Appropriate Approach: **Deep Reinforcement Learning (DRL) with DQN**
- **Why DQN?**: DQN can handle large state spaces by approximating the Q-value function using a neural network. It does not require a precise model of the environment and can learn optimal policies through interaction.
- **Advantages**: DQN can scale to high-dimensional state representations, making it suitable for real-world traffic signal control where the state space might include additional features (e.g., traffic density, time of day).
- **Implementation**: The agent can be trained using simulation environments or historical traffic data to learn the optimal policy for switching traffic signals.

By using DRL with DQN, we can effectively manage the complexity of the state space and learn a robust policy for controlling traffic signals at a zebra crossing, minimizing waiting times for both pedestrians and vehicles.

(d)

Multiplying all the rewards in a Markov Decision Process (MDP) by a constant factor, such as 10, affects the magnitude of the rewards but does not change the optimal policy. Let's explore the impact of this change in detail:

Bellman Equation with Scaled Rewards

The Bellman state-value equation with rewards scaled by a factor of 10 can be written as:

$$ V'(S) = \max_{A} \left[10 \cdot R(S, A) + \gamma \sum_{S'} P(S'|S, A) V'(S') \right] $$

where $V'(S)$ is the new value function with the scaled rewards.

Impact on the Value Function

If we denote the original value function by $V(S)$, the relationship between $V(S)$ and $V'(S)$ can be derived as follows. The original Bellman equation is:

$$V(S) = \max_{A} \left[R(S, A) + \gamma \sum_{S'} P(S'|S, A) V(S') \right]$$

With the rewards scaled by 10, the new equation becomes:

$$V'(S) = \max_{A} \left[10 \cdot R(S, A) + \gamma \sum_{S'} P(S'|S, A) V'(S') \right]$$

Relationship Between $V'(S)$ and $V(S)$

Let's assume that $V'(S) = 10 \cdot V(S)$. Substituting this into the new Bellman equation:

$$10 \cdot V(S) = \max_{A} \left[10 \cdot R(S, A) + \gamma \sum_{S'} P(S'|S, A) 10 \cdot V(S') \right]$$

$$10 \cdot V(S) = \max_{A} \left[10 \cdot (R(S, A) + \gamma \sum_{S'} P(S'|S, A) V(S')) \right]$$

$$V(S) = \max_{A} \left[R(S, A) + \gamma \sum_{S'} P(S'|S, A) V(S') \right]$$

This confirms that if $V'(S) = 10 \cdot V(S)$, the Bellman equations for $V(S)$ and $V'(S)$ are consistent.

Impact on the Optimal Policy

The optimal policy is derived by selecting actions that maximize the value function. Since multiplying the rewards by a constant factor does not change the relative differences between the rewards of different actions, the optimal policy remains unchanged.

Formally, if $\pi(S)$ is the optimal policy under the original rewards, it is given by:

$$\pi(S) = \arg\max_{A} \left[R(S, A) + \gamma \sum_{S'} P(S'|S, A) V(S') \right]$$

With the scaled rewards, the optimal policy $\pi'(S)$ is:

$$\pi'(S) = \arg\max_{A} \left[10 \cdot R(S, A) + \gamma \sum_{S'} P(S'|S, A) V'(S') \right]$$

Since $V'(S) = 10 \cdot V(S)$, we have:

$$\pi'(S) = \arg\max_{A} \left[10 \cdot (R(S, A) + \gamma \sum_{S'} P(S'|S, A) V(S')) \right]$$

$$\pi'(S) = \arg\max_{A} \left[R(S, A) + \gamma \sum_{S'} P(S'|S, A) V(S') \right]$$

Thus, $\pi'(S) = \pi(S)$.

Summary

- **Value Function Scaling**: The value function $V'(S)$ under scaled rewards is exactly 10 times the original value function $V(S)$.
- **Optimal Policy**: The optimal policy remains unchanged because the relative ordering of action values is preserved.

Multiplying all rewards by a constant factor affects the scale of the value function but does not change the optimal policy. The agent's decision-making process remains the same, ensuring that the same actions are selected in each state to maximize the long-term rewards.

Q2) Considering Netflix's personalised video recommendation for an individual, where Netflix suggests a genre from the available list of genres ($\in \{a, b, c, d\}$) and observes the reward (satisfactory rating) $\in \{0, 1, 2, 3\}$ from the individual, let's analyse the following sequence of tried genres and rewards: 1: (a,2), 2: (b,3), 3: (c,1), 4: (a,1), 5: (b,1), 6: (d,2), 7: (b,1), 8: (d,1), 9: (c,3), 10: (c,3), 11: (c,1), 12: (d,1). Assuming the initial value of all the actions are the same as the last digit of your student id divided by 2, please provide answers to the following questions in this specific context. [DRL Exam Question 2 ; 22-July-2023]

(a) (Stationary case) Which of those suggestions are exploratory in nature assuming ε-greedy action selection and action values are obtained by sample averages ? Why? [1.5 M]

(b) (Non-Stationary case)Which of those suggestions are exploratory in nature assuming ε-greedy action selection and action values are obtained by weighted averages with $\alpha = 0.5$? Why? [1.5 M]

(c) Compare your answers to (a) and (b) and explain the difference, ignoring the assumption on stationarity. [1.5 M]

(d) If we initialise all the action values to be 5, will the exploratory moves due to ε-greedy behave differently? Explain. [1.5 M]

(e) Assuming there are n-individuals whose likings on movies are different. How would you extend the solution based on armed bandits in this case? [1.5 M]

Answer:

(a)

To determine which suggestions are exploratory in nature under the ε-greedy action selection strategy, we first need to understand the ε-greedy method and calculate the action values using sample averages.

In the ε-greedy strategy:
- With probability $(1 - \varepsilon)$, the action with the highest estimated value (greedy action) is selected.
- With probability ε, a random action (exploratory action) is selected.

We assume ε is given but since it isn't specified, we will describe the method of identifying exploratory actions.

Calculating Sample Averages for Action Values

Let's define $Q(a)$, $Q(b)$, $Q(c)$, and $Q(d)$ as the estimated values for actions a, b, c, and d, respectively.

Given sequence of trials and rewards:
1. $(a,2)$
2. $(b,3)$
3. $(c,1)$
4. $(a,1)$
5. $(b,1)$
6. $(d,2)$
7. $(b,1)$
8. $(d,1)$
9. $(c,3)$
10. $(c,3)$
11. $(c,1)$
12. $(d,1)$

We compute the sample average for each action (genre):

For action a:
- Occurrences: 2
- Rewards: $[2, 1]$
- Average: $Q(a) = \frac{2+1}{2} = 1.5$

For action b:
- Occurrences: 3
- Rewards: $[3, 1, 1]$
- Average: $Q(b) = \frac{3+1+1}{3} = 1.67$

For action c:
- Occurrences: 4
- Rewards: $[1, 3, 3, 1]$
- Average: $Q(c) = \frac{1+3+3+1}{4} = 2$

For action d:
- Occurrences: 3
- Rewards: $[2, 1, 1]$
- Average: $Q(d) = \frac{2+1+1}{3} = 1.33$

Identifying Exploratory Actions

Assume the initial value for each action is the last digit of your student ID divided by 2. For the sake of example, let’s say the last digit of the student ID is 8. Thus, the initial value of each action is $\frac{8}{2} = 4$.

Given the action values (sample averages):
- $Q(a) = 1.5$

- $Q(b) = 1.67$
- $Q(c) = 2$
- $Q(d) = 1.33$

To identify which suggestions are exploratory, we need to see if they were not the greedy choice at the time of selection.

Here's the sequence again with decisions:

1. $(a,2)$: Initial values are all the same (4). Any choice is exploratory.
2. $(b,3)$: Initial values are all the same (4). Any choice is exploratory.
3. $(c,1)$: Initial values are all the same (4). Any choice is exploratory.
4. $(a,1)$: Now we have $Q(a)=2$, $Q(b)=3$, $Q(c)=1$. $Q(b)$ is highest, choosing a is exploratory.
5. $(b,1)$: Now we have $Q(a)=1.5$, $Q(b)=3$, $Q(c)=1$. $Q(b)$ is highest, choosing b is greedy.
6. $(d,2)$: Now we have $Q(a)=1.5$, $Q(b)=2$, $Q(c)=1$. $Q(b)$ is highest, choosing d is exploratory.
7. $(b,1)$: Now we have $Q(a)=1.5$, $Q(b)=2$, $Q(c)=1$, $Q(d)=2$. $Q(b)$ or $Q(d)$ is highest, choosing b is greedy.
8. $(d,1)$: Now we have $Q(a)=1.5$, $Q(b)=1.67$, $Q(c)=1$, $Q(d)=2$. $Q(d)$ is highest, choosing d is greedy.
9. $(c,3)$: Now we have $Q(a)=1.5$, $Q(b)=1.67$, $Q(c)=1$, $Q(d)=1.5$. $Q(b)$ is highest, choosing c is exploratory.
10. $(c,3)$: Now we have $Q(a)=1.5$, $Q(b)=1.67$, $Q(c)=2$, $Q(d)=1.5$. $Q(c)$ is highest, choosing c is greedy.
11. $(c,1)$: Now we have $Q(a)=1.5$, $Q(b)=1.67$, $Q(c)=3$, $Q(d)=1.5$. $Q(c)$ is highest, choosing c is greedy.
12. $(d,1)$: Now we have $Q(a)=1.5$, $Q(b)=1.67$, $Q(c)=2$, $Q(d)=1.33$. $Q(c)$ is highest, choosing d is exploratory.

Summary of Exploratory Actions

The exploratory actions in the sequence are:
1. $(a,2)$
2. $(b,3)$
3. $(c,1)$
4. $(a,1)$
6. $(d,2)$
9. $(c,3)$
12. $(d,1)$

These actions were not the highest estimated value at the time of their selection, assuming the ε-greedy strategy where some probability is allocated to exploration (choosing a non-greedy action).

(b)

In the non-stationary case, we use a weighted average (also called an exponential moving average) to update the action values with a constant step-size parameter $\alpha = 0.5$. The update rule for the action value Q after taking action a and receiving reward r is:

$$ Q(a) \leftarrow Q(a) + \alpha \left(r - Q(a) \right) $$

Calculating Weighted Averages for Action Values

Let's apply the update rule to compute the action values step-by-step for the given sequence of trials and rewards:

Initialization
Assume the initial value for each action is the last digit of your student ID divided by 2. For the sake of example, let’s say the last digit of the student ID is 8. Thus, the initial value of each action is 4.

1. **(a, 2)**:
   $$
   Q(a) = 4 + 0.5 \times (2 - 4) = 4 + 0.5 \times (-2) = 4 - 1 = 3
   $$
2. **(b, 3)**:
   $$
   Q(b) = 4 + 0.5 \times (3 - 4) = 4 + 0.5 \times (-1) = 4 - 0.5 = 3.5
   $$
3. **(c, 1)**:
   $$
   Q(c) = 4 + 0.5 \times (1 - 4) = 4 + 0.5 \times (-3) = 4 - 1.5 = 2.5
   $$
4. **(a, 1)**:
   $$
   Q(a) = 3 + 0.5 \times (1 - 3) = 3 + 0.5 \times (-2) = 3 - 1 = 2
   $$
5. **(b, 1)**:
   $$
   Q(b) = 3.5 + 0.5 \times (1 - 3.5) = 3.5 + 0.5 \times (-2.5) = 3.5 - 1.25 = 2.25
   $$
6. **(d, 2)**:
   $$
   Q(d) = 4 + 0.5 \times (2 - 4) = 4 + 0.5 \times (-2) = 4 - 1 = 3
   $$
7. **(b, 1)**:
   $$
   Q(b) = 2.25 + 0.5 \times (1 - 2.25) = 2.25 + 0.5 \times (-1.25) = 2.25 - 0.625 = 1.625
   $$
8. **(d, 1)**:
   $$
   Q(d) = 3 + 0.5 \times (1 - 3) = 3 + 0.5 \times (-2) = 3 - 1 = 2
   $$
9. **(c, 3)**:
   $$

$$
Q(c) = 2.5 + 0.5 \times (3 - 2.5) = 2.5 + 0.5 \times (0.5) = 2.5 + 0.25 = 2.75
$$

10. **(c, 3)**:
$$
Q(c) = 2.75 + 0.5 \times (3 - 2.75) = 2.75 + 0.5 \times (0.25) = 2.75 + 0.125 = 2.875
$$

11. **(c, 1)**:
$$
Q(c) = 2.875 + 0.5 \times (1 - 2.875) = 2.875 + 0.5 \times (-1.875) = 2.875 - 0.9375 = 1.9375
$$

12. **(d, 1)**:
$$
Q(d) = 2 + 0.5 \times (1 - 2) = 2 + 0.5 \times (-1) = 2 - 0.5 = 1.5
$$

Identifying Exploratory Actions

To identify which suggestions are exploratory, we need to determine whether the chosen action at each step was not the action with the highest estimated value at that time.

Let's list the action values and the decisions:

1. $(a, 2)$: Initial values all 4. Any choice is exploratory.
2. $(b, 3)$: Initial values all 4. Any choice is exploratory.
3. $(c, 1)$: Initial values all 4. Any choice is exploratory.
4. $(a, 1)$: Now we have $Q(a)=3$, $Q(b)=3.5$, $Q(c)=2.5$, $Q(d)=4$. $Q(d)$ is highest, choosing a is exploratory.
5. $(b, 1)$: Now we have $Q(a)=2$, $Q(b)=3.5$, $Q(c)=2.5$, $Q(d)=4$. $Q(d)$ is highest, choosing b is exploratory.
6. $(d, 2)$: Now we have $Q(a)=2$, $Q(b)=2.25$, $Q(c)=2.5$, $Q(d)=4$. $Q(d)$ is highest, choosing d is greedy.
7. $(b, 1)$: Now we have $Q(a)=2$, $Q(b)=2.25$, $Q(c)=2.5$, $Q(d)=3$. $Q(d)$ is highest, choosing b is exploratory.
8. $(d, 1)$: Now we have $Q(a)=2$, $Q(b)=1.625$, $Q(c)=2.5$, $Q(d)=3$. $Q(d)$ is highest, choosing d is greedy.
9. $(c, 3)$: Now we have $Q(a)=2$, $Q(b)=1.625$, $Q(c)=2.5$, $Q(d)=2$. $Q(a)$ or $Q(c)$ is highest, choosing c is greedy.
10. $(c, 3)$: Now we have $Q(a)=2$, $Q(b)=1.625$, $Q(c)=2.75$, $Q(d)=2$. $Q(c)$ is highest, choosing c is greedy.
11. $(c, 1)$: Now we have $Q(a)=2$, $Q(b)=1.625$, $Q(c)=2.875$, $Q(d)=2$. $Q(c)$ is highest, choosing c is greedy.
12. $(d, 1)$: Now we have $Q(a)=2$, $Q(b)=1.625$, $Q(c)=1.9375$, $Q(d)=2$. $Q(d)$ is highest, choosing d is greedy.

Summary of Exploratory Actions

The exploratory actions in the sequence are:
1. $(a, 2)$
2. $(b, 3)$

3. \((c, 1) \)
4. \((a, 1) \)
5. \((b, 1) \)
7. \((b, 1) \)

These actions were not the highest estimated value at the time of their selection, assuming the ε-greedy strategy where some probability is allocated to exploration (choosing a non-greedy action).

(c)

In comparing the answers to parts (a) and (b), we need to understand how the method of updating action values (sample averages vs. weighted averages) impacts the identification of exploratory actions under the ε-greedy strategy. Here's a detailed comparison:

Part (a): Stationary Case with Sample Averages

In this scenario, the action values are updated using sample averages, meaning each action's value is the average of all observed rewards for that action up to that point. The exploratory actions identified are:

1. \((a, 2) \)
2. \((b, 3) \)
3. \((c, 1) \)
4. \((a, 1) \)
6. \((d, 2) \)
9. \((c, 3) \)
12. \((d, 1) \)

Part (b): Non-Stationary Case with Weighted Averages (\(\alpha = 0.5\))

In this scenario, the action values are updated using weighted averages, meaning each new reward has a significant and fixed impact on the action value. The exploratory actions identified are:

1. \((a, 2) \)
2. \((b, 3) \)
3. \((c, 1) \)
4. \((a, 1) \)
5. \((b, 1) \)
7. \((b, 1) \)

Comparison and Explanation

1. **Initial Steps**:
   - In both scenarios, the first three actions \((a, 2)\), \((b, 3)\), and \((c, 1)\) are exploratory because the initial values are all equal, and any choice is effectively exploratory.
   - The action \((a, 1)\) is also exploratory in both cases because the updated values after the first three actions lead to a situation where action \(a\) is not the highest valued action.

2. **Subsequent Differences**:
   - In the stationary case with sample averages, the exploratory actions identified are based on the long-term average rewards. This method results in identifying actions \((d, 2)\), \((c, 3)\), and \((d, 1)\) as exploratory because, at those points, the chosen actions were not the highest based on the accumulated average rewards.
   - In the non-stationary case with weighted averages, actions \((b, 1)\) and \((b, 1)\) are identified as exploratory. This happens because the action values are more volatile due to the fixed step-size parameter (\(\alpha = 0.5\)). Each reward significantly impacts the estimated action values, making the selection process more sensitive to recent rewards.

3. **Impact of Action Value Update Methods**:
   - **Sample Averages**: This method smooths out the impact of individual rewards over time, providing a more stable estimate of the action values. This stability means that actions are less likely to be exploratory once sufficient data has been collected unless the data inherently supports exploration.
   - **Weighted Averages**: This method gives more weight to recent rewards, making the action values more responsive to the latest feedback. As a result, the estimated action values can fluctuate more, leading to a higher likelihood of exploratory actions being chosen as the system reacts to recent changes in rewards.

Summary of Differences

- **Stationary Case**: The action values converge more slowly, and exploration occurs based on long-term averages. The identified exploratory actions reflect the method's gradual learning process.
- **Non-Stationary Case**: The action values adjust quickly to recent rewards, leading to more frequent updates and a higher likelihood of exploratory actions based on the immediate reward changes.

Ignoring the assumption on stationarity, the primary difference arises from how action values are updated: the sample average method results in more stable estimates, while the weighted average method causes the system to be more adaptive to recent changes, hence more explorative in nature.

(d)

Initializing all the action values to a higher value like 5 will indeed affect the behavior of the ε-greedy algorithm in both the stationary and non-stationary cases. Here’s how and why:

Impact on ε-greedy Behavior

Higher Initial Values

When all action values are initialized to a high value (e.g., 5), it creates an optimistic bias. This means that initially, all actions appear highly attractive. This optimistic initialization encourages exploration, especially in the beginning, because even suboptimal actions seem promising initially.

Stationary Case with Sample Averages

1. **Initial Exploration**: With high initial values (5), the first few actions are exploratory because the algorithm will choose randomly among actions with the same high initial value.

2. **Convergence**: As rewards are observed and averaged, the action values will start to reflect the true expected rewards. The initial high values will decay towards the true averages based on the rewards received.
3. **Exploratory Actions**: Due to the high initial values, more exploration will occur early on until the true values are learned. Once the values stabilize, the algorithm will behave more greedily, choosing the action with the highest average reward.

Non-Stationary Case with Weighted Averages (α = 0.5)

1. **Initial Exploration**: Similarly, with high initial values (5), the first few actions are exploratory due to random choice among initially equally attractive actions.
2. **Rapid Adjustment**: The weighted average update rule ($Q(a) \leftarrow Q(a) + \alpha (r - Q(a))$) will quickly adjust the action values towards the recent rewards. However, because of the high initial values, the initial adjustments will lead to significant drops from the initial value.
3. **Exploratory Actions**: High initial values will cause the action values to decrease rapidly initially as rewards are observed. This rapid decrease may lead to a mix of exploratory and greedy actions depending on how quickly the values adjust to reflect the true expected rewards.

Example Sequence Analysis with Initial Value 5

Consider the same sequence of trials and rewards:

1. **(a, 2)**:
   $$Q(a) = 5 + 0.5 \times (2 - 5) = 5 + 0.5 \times (-3) = 5 - 1.5 = 3.5$$
2. **(b, 3)**:
   $$Q(b) = 5 + 0.5 \times (3 - 5) = 5 + 0.5 \times (-2) = 5 - 1 = 4$$
3. **(c, 1)**:
   $$Q(c) = 5 + 0.5 \times (1 - 5) = 5 + 0.5 \times (-4) = 5 - 2 = 3$$
4. **(a, 1)**:
   $$Q(a) = 3.5 + 0.5 \times (1 - 3.5) = 3.5 + 0.5 \times (-2.5) = 3.5 - 1.25 = 2.25$$
5. **(b, 1)**:
   $$Q(b) = 4 + 0.5 \times (1 - 4) = 4 + 0.5 \times (-3) = 4 - 1.5 = 2.5$$
6. **(d, 2)**:
   $$Q(d) = 5 + 0.5 \times (2 - 5) = 5 + 0.5 \times (-3) = 5 - 1.5 = 3.5$$
7. **(b, 1)**:
   $$Q(b) = 2.5 + 0.5 \times (1 - 2.5) = 2.5 + 0.5 \times (-1.5) = 2.5 - 0.75 = 1.75$$

\]
8. **(d, 1)**:
\[
Q(d) = 3.5 + 0.5 \times (1 - 3.5) = 3.5 + 0.5 \times (-2.5) = 3.5 - 1.25 = 2.25
\]
9. **(c, 3)**:
\[
Q(c) = 3 + 0.5 \times (3 - 3) = 3
\]
10. **(c, 3)**:
\[
Q(c) = 3 + 0.5 \times (3 - 3) = 3
\]
11. **(c, 1)**:
\[
Q(c) = 3 + 0.5 \times (1 - 3) = 3 + 0.5 \times (-2) = 3 - 1 = 2
\]
12. **(d, 1)**:
\[
Q(d) = 2.25 + 0.5 \times (1 - 2.25) = 2.25 + 0.5 \times (-1.25) = 2.25 - 0.625 = 1.625
\]

Comparison and Summary

- **Initial Bias**: High initial values (5) lead to optimistic initial estimates, encouraging exploration early on.
- **Decay towards True Values**: In both cases, the high initial values will decay towards the true values of the rewards over time.
- **More Exploration Early On**: The high initial values result in more exploratory actions initially as the ε-greedy policy selects among actions with the same optimistic initial values.
- **Convergence**: In the stationary case, the values stabilize based on the average rewards, while in the non-stationary case, the values adjust more dynamically due to the weighted averaging.

By initializing with high values, the ε-greedy strategy is nudged to explore more initially, which can be beneficial in discovering the optimal actions sooner. The difference in behavior due to initial values highlights the importance of exploration in reinforcement learning algorithms.

(e)

To extend the solution to accommodate \(n\)-individuals with different likings on movies, we can employ a multi-armed bandit approach tailored for multiple users. The challenge lies in effectively learning and recommending movies to each individual while considering the diversity in their preferences. Here's a detailed strategy for this extension:

Personalized Multi-Armed Bandit Approach

1. **Separate Bandit for Each User**:
   - Maintain a separate multi-armed bandit for each individual user. Each bandit instance will independently learn the preferences of the user it is associated with.

2. **Contextual Bandits**:
   - Use contextual bandits where the context is the identity of the user. This way, the bandit algorithm can take into account the specific preferences of each user.

Steps for Implementation

1. **Initialization**:
   - Initialize the action values for each genre for each user. Optimistic initial values (e.g., setting all values to 5) can encourage exploration.

2. **Action Selection (ε-greedy)**:
   - For each user, apply the ε-greedy strategy to select a genre. With probability \(ε\), choose a random genre (exploration). With probability \(1-ε\), choose the genre with the highest estimated value (exploitation).

3. **Reward Observation**:
   - Observe the reward (user's rating) for the selected genre.

4. **Action Value Update**:
   - Update the estimated value of the selected genre for the user based on the observed reward. This can be done using:
     - **Sample Averages**: For stationary environments.
     - **Weighted Averages**: For non-stationary environments (e.g., using \(\alpha = 0.5\) for exponential moving averages).

5. **Iterate**:
   - Repeat the process for each user and each recommendation cycle, continuously refining the estimated preferences.

Algorithm Example

Here’s a pseudo-code example of how this can be implemented:

Considerations and Enhancements

1. **User Segmentation**:
   - Group similar users and share learning among them to accelerate convergence.

2. **Cold Start Problem**:
   - Use initial surveys or collaborative filtering to set better initial values.

3. **Contextual Features**:
   - Incorporate contextual information (e.g., time of day, recent watch history) to make the bandit more responsive to changing user preferences.

4. **Scalability**:
   - Ensure the system can handle a large number of users and genres efficiently, possibly leveraging parallel processing or distributed computing.

5. **Hybrid Approaches**:
   - Combine bandit algorithms with recommendation systems such as collaborative filtering or content-based filtering to improve performance.

By maintaining separate bandits for each user and utilizing contextual information, we can create a personalized recommendation system that adapts to the unique preferences of each individual, leveraging the strengths of multi-armed bandit algorithms in a multi-user setting.

Q3) Consider a robot navigating in an environment consisting of six positions,numbered from 0 to 5.

0	1	2	3	4	5

At each location, the robot has two actions available: moving left (⇦) and moving right (⇒). These actions are selected with equal probability. States 0 and 5 are considered terminal states, meaning no further actions are permitted from there. When the agent transitions into state 5, it receives a reward of +x and when the agent transitions into state 0, it receives a reward of -x. *x is determined by the last digit of your student ID plus 5*. For all other transitions, the agent receives a reward of 0.

Answer the following questions in this context. Use in-place version (asynchronous dynamic programming) of the algorithms. The discount factor, γ is 0.5. [DRL Exam Question 3 ; 22-July-2023]

(a) Consider the values initialised to 0's. Evaluate the following policy. Use your answer to improve the policy (i.e. one iteration). Show all the steps. [3 M]

	⇦	⇦	⇦	⇒	

(b) Consider the values initialised to 0's. Show two iterations of value iteration. Show all the steps. [3 M]

(c) Comment on the applicability of both the algorithms (a) & (b) for problems with large state and action spaces in no more than 3 well articulated statements. [1.5 M]

Answer:

(a)

(b)

(c)